GLIMMER: generalized late-interaction memory reranker

Michiel de Jong[*] [†], **Yury Zemlyanskiy**[*]
Nicholas FitzGerald, **Fei Sha**, **Sumit Sanghai**, **William W. Cohen**, **Joshua Ainslie**

Google Research

arXiv:2306.10231v1 [cs.CL] 17 Jun 2023

Abstract

Memory augmentation is a powerful approach for efficiently incorporating external information into language models, but leads to reduced performance relative to retrieving text. Recent work introduced LUMEN, a memory-retrieval hybrid that partially pre-computes memory and updates memory representations on the fly with a smaller live encoder.

We propose GLIMMER, which improves on this approach through 1) exploiting free access to the powerful memory representations by applying a shallow reranker on top of memory to drastically improve retrieval quality at low cost, and 2) incorporating multi-task training to learn a general and higher quality memory and live encoder. GLIMMER achieves strong gains in performance at faster speeds compared to LUMEN and FiD on the KILT benchmark of knowledge-intensive tasks.

1 Introduction

Retrieval-augmented language models achieve strong performance, but are computationally expensive due to the need to process retrieved passages. A large body of work attempts to reduce the cost of reading retrieved passages through conditional computation (Ainslie et al., 2023b; Varshney et al., 2022; Schuster et al., 2022), reranking (Wang et al., 2018; Yu et al., 2022; Wang et al., 2018), or memory (de Jong et al., 2022b; Wu et al., 2022a; Li et al., 2022).

Reranking improves retrieval quality and therefore reduces the number of passages that need to be processed by the reader. However, neural reranking is expensive, as each retrieved candidate is processed by a neural network. Late interaction rerankers (Khattab and Zaharia, 2020; Cohen et al., 2022; MacAvaney et al., 2020) pre-compute intermediate token representations and apply a smaller neural model on the fly to combine query and document representations and produce a ranking score. Late interaction drastically improves speed at the cost of storage and pre-computation overhead and machinery.

Recently the idea of late-interaction has also been applied to retrieval augmented generation: LUMEN (de Jong et al., 2023) interpolates between memory and retrieval augmentation to achieve a better quality-compute trade-off.

We propose GLIMMER (Generalized Late-Interaction Memory Reranker), a late interaction approach that combines these lines of work by *unifying reranking and memory into a single end-to-end model*. Like LUMEN, GLIMMER consists of a memory encoder that generates pre-computed token representations for retrieval documents, and a live encoder that combines the representations of retrieved documents with the query. After the first layers of the live-encoder, a ranking layer selects the most relevant passages which are retained for further processing. The model is trained to rank passages by usefulness to the reader through a perplexity distillation auxiliary loss (Izacard et al., 2022).

GLIMMER also improves on LUMEN by using a single general memory and live encoder over all tasks, trained with multi-task fine-tuning over knowledge intensive datasets.

We evaluate on the KILT benchmark of knowledge-intensive tasks (Petroni et al., 2020). We first find that multi-task training of the memory and live encoders strongly improves model quality relative to training on a single task, especially when devoting less capacity to the live encoder. Moreover, GLIMMER strongly improves over both multi-task trained LUMEN and FiD in both quality and speed. In general, GLIMMER successfully unifies reranking and memory into a single efficient, high-quality model.

[*] Equal contribution.
[†] University of Southern California. Work done at Google Research.

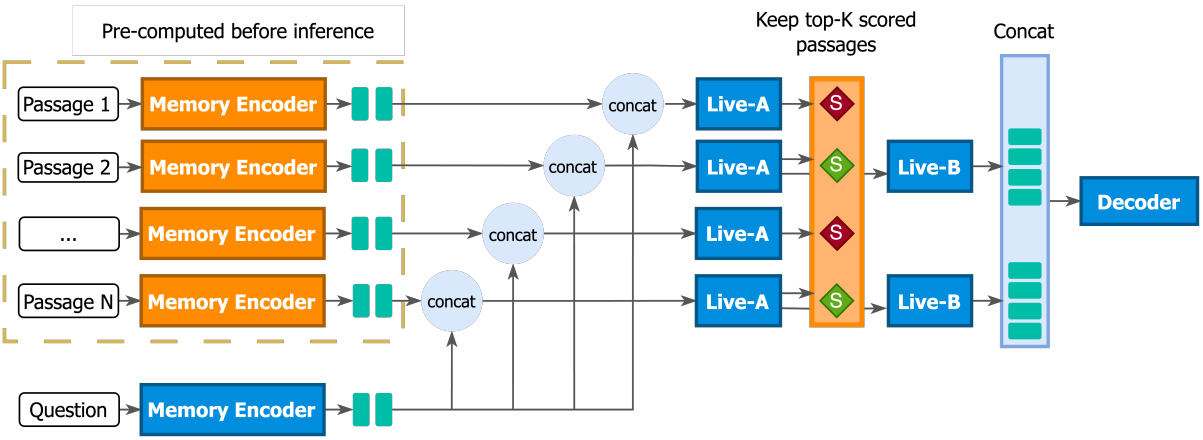

Figure 1: Overview of GLIMMER architecture.
Memory: The memory encoder is updated during multi-task training, unlike LUMEN, before being applied to the corpus to generate partially pre-computed memory representations. The memory encoder is also applied during inference to generate partial question representations that are compatible with the memory.
Live: Each passage memory is concatenated with the question representation, and a live encoder (proportion α of the total model) is then applied to condition the passage on the input in two stages. After the first stage, consisting of a fraction β of live layers, a scoring layer selects a small subset of high-scoring relevant passages to keep and less relevant passages are discarded. The selected passage representations are updated by the second stage of the live encoder. Finally, the conditioned representations are concatenated and attended to by the decoder as in FiD.

2 Background

We are interested in achieving the best possible trade-off between quality and inference compute. The following section describes FiD and LUMEN, the baseline methods that GLIMMER is built on, and their computational properties. A more in-depth analysis of these methods can be found in de Jong et al. (2023).

2.1 Fusion-in-Decoder

Fusion-in-Decoder (Izacard and Grave, 2021) is based on a T5 encoder-decoder model (Raffel et al., 2020). For each input, a number of relevant text passages are retrieved, and the input is prepended to each passage. The resulting input-passage pairs are encoded separately by the encoder, and the encoded pairs are then concatenated into a flat sequence of token representations and attended to by the decoder to produce a target output. For each model, **live** components are in blue and components **pre-computed** before inference in orange.

$$G = \textbf{Dec}\Big[\textbf{Enc}(Q; \text{Passage}_1); \ldots \textbf{Enc}(Q; \text{Passage}_k)\Big]$$

Let k be the number of passages, n_p be the number of tokens per passage, n_t the number of target tokens, L the number of layers, and d the dimension of the model. Following analysis from de Jong et al. (2022a, 2023), the FLOPs for a single inference sample of FiD (ignoring attention score computation) is given by

$$F_{FiD} = \underbrace{kn_p \cdot L \cdot 14d^2}_{\text{Encoder and cross-attention}} + \underbrace{n_t \cdot L \cdot 14d^2}_{\text{Decoder}}$$

with factors $8d^2$ per token from feedforward layers, $4d^2$ from self-attention projection layers, and $2d^2$ from cross-attention projection layers. de Jong et al. (2023) contains a derivation of FiD model complexity in greater detail.

2.2 LUMEN

Typically the combined length of retrieved passages is much larger than the target length, such that the majority of FLOPs are consumed by the encoder processing retrieved passages. LUMEN reduces encoder inference cost by partially pre-computing the encoder representation for retrieved passages. At inference time, LUMEN retrieves the intermediate layer representations rather than the text.

More precisely, LUMEN is initialized from a pre-trained T5 encoder-decoder model. The decoder functions the same as the standard FiD decoder, but the T5 encoder is divided into a large **memory encoder** which contains the first $1 - \alpha$ proportion of layers, and a smaller **live encoder** with the remaining α proportion of layers. The memory encoder is applied offline to passages in the corpus to pre-compute memory representations, which are later updated conditioned on input and task on the fly by

the fine-tuned live encoder. In order to ensure that memory representations and input are compatible, LUMEN applies the memory encoder[1] to the input before prepending the question representation to the memory representation.

$$H_i = \Big[\textbf{MemEnc}(Q);\ \ \textbf{MemEnc}(\text{Passage}_i)\Big]$$
$$G = \textbf{Dec}\Big[Q; \textbf{LiveEnc}(H_1); \ldots \textbf{LiveEnc}(H_k)\Big]$$

Choosing $\alpha = 1$ yields a model very close to FiD while $\alpha = 0$ is a full memory model. During inference LUMEN applies only a proportion α of the layers, leading to a fraction α of FiD reader FLOPs for any given model size.

$$F_{\text{LUMEN}} = \underbrace{kn_p \cdot \alpha L \cdot 12d^2}_{\text{Encoder}} + \underbrace{kn_p \cdot L \cdot 2d^2}_{\text{Cross-attention}} + \underbrace{n_t \cdot L \cdot 14d^2}_{\text{Decoder}}$$

3 GLIMMER

GLIMMER builds on LUMEN with two major differences: GLIMMER incorporates a built-in reranker, and shares the memory and live encoder across many tasks. Standard reranking approaches struggle with a trade-off: smaller models may not be sufficiently powerful to judge whether a passage is relevant to an input, while the cost of larger models defeats a large part of the purpose of using a reranker in the first place. The LUMEN architecture offers an opportunity to circumvent this trade-off, as the majority of the passage representations are pre-computed. GLIMMER re-uses the initial layers of the live encoder for reranking, yielding a powerful re-ranking model at relatively modest computational cost.

Sharing weights across tasks, meanwhile, allows for training the memory encoder without storing duplicate pre-computed representations, and strongly increases the effectiveness of the live encoder. Figure 1 shows an overview of the GLIMMER architecture.

3.1 Architecture

Compared to LUMEN, GLIMMER divides the live encoder into two components, where the first component is responsible for initial interaction and reranking and the second component performs further processing on representations of selected passages. The first component contains β proportion of live encoder layers with the remainder of layers in the second component. After the first live encoder, a linear projection layer is applied to the first token of each input-passage pair to generate a relevance score for the passage. The top-m passages with the highest scores out of the original k are processed by the second live encoder, and the other passages are discarded. The output of the second live encoder is fed to the decoder as in FiD and LUMEN.

$$H_i = \Big[\textbf{MemEnc}(Q);\ \ \textbf{MemEnc}(\text{Passage}_i)\Big]$$
$$H'_i = \textbf{LiveEncA}(H_i)$$
$$R_j = H'_i \ \text{ s.t. Rank } [\textbf{Score}(H'_i)] = j$$
$$G = \textbf{Dec}\Big[Q; \textbf{LiveEncB}(R_1); \ldots \textbf{LiveEncB}(R_m)\Big]$$

3.2 Training

The memory encoder, both live encoder components, the scoring projection and the decoder are all trained end-to-end. Unlike in LUMEN, the memory encoder does not need to be frozen as we share a single memory encoder between all tasks. In order to train the scoring projection and encourage the memory and first live encoder to produce representations suitable for reranking, we employ an auxiliary perplexity distillation loss (Izacard et al., 2022). This loss encourages the model to rank passages by how much they lower the perplexity of the final generation, if that input-passage was fed to the decoder by itself. In particular, perplexity distillation minimizes the KL-divergence between the distribution implied by the reranking scores (computed from the output of the first live encoder component applied to concatenation of input and passage representations) and the distribution implied by the resulting perplexities:

$$p_k^{\text{rank}} = \frac{\exp(\text{Score}(\text{Passage}_k, Q)/\tau)}{\sum_i \exp(\text{Score}(\text{Passage},_i, Q)/\tau)}$$

$$p_k^{\text{LM}} = \frac{\exp(\log p_{LM}(\text{Answer}|\text{Passage}_k, Q)/\tau)}{\sum_i \exp(\log p_{LM}(\text{Answer}|\text{Passage}_i, Q)/\tau)}$$

$$\mathcal{L}_{\text{pdist}} = KL(p^{\text{rank}},\ p^{\text{LM}})$$

3.3 Computational analysis

The difference in computational complexity between GLIMMER and LUMEN lies in reranking. The m selected passages are processed by the entire live encoder and then fed through the decoder, yielding

[1] The original LUMEN implementation used a separate question encoder, but we show this is unnecessary.

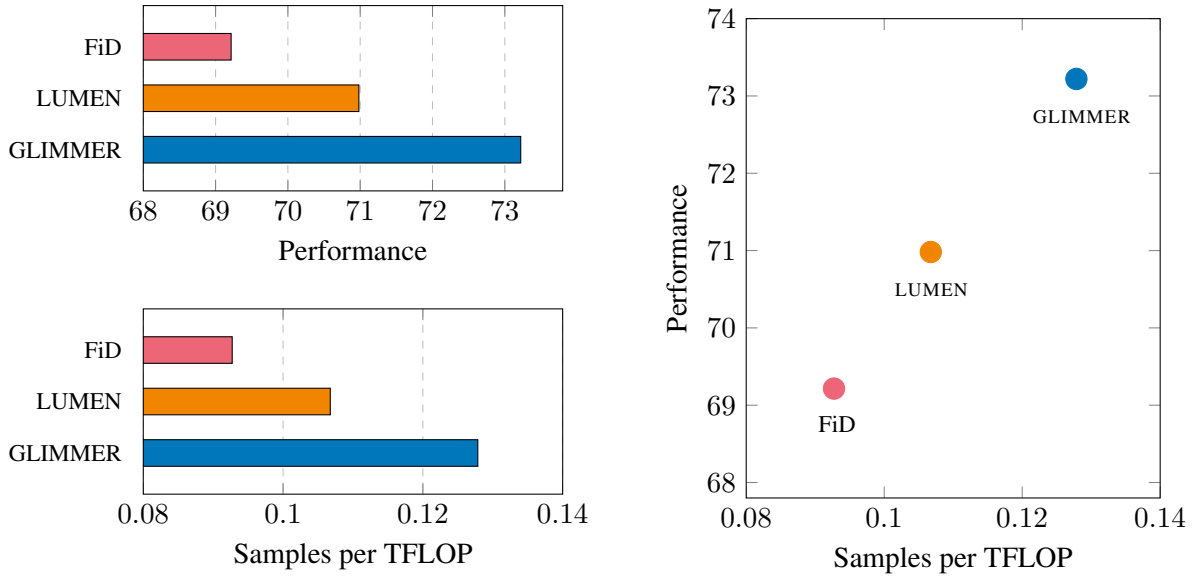

Figure 2: **GLIMMER is faster and higher quality than LUMEN which in turn is faster and higher quality than FiD.** Comparison of GLIMMER, LUMEN and FiD XXL model average performance on KILT dev set, and inference speed. FiD uses 5 retrieved passages, LUMEN uses 10 retrieved passages, and GLIMMER uses 25 retrieved passages, reranked to 5 final passages. LUMEN and GLIMMER have live proportion $\alpha = \frac{1}{3}$.

computational cost equal to applying LUMEN with m passages (less than the full number of retrieved passages k). However, for the passages that were not selected, GLIMMER still applied the first live encoder component, leading to a reranking cost:

$$F_{\text{GLIMMER}} = F^{m}_{\text{LUMEN}} + \underbrace{(k-m)n_p \cdot \beta\alpha L \cdot 12d^2}_{\text{Reranking}}$$

If we use a small number of selected passages $m << k$ and small fraction of reranking layers $\beta << 1$, then GLIMMER is significantly less computationally intensive than LUMEN. with k retrievals.

We note that this computational analysis is limited to FLOPs, rather than practical latency. For autoregressive inference, the decoder is often bottlenecked by memory bandwidth rather than FLOPs (Shazeer, 2019; de Jong et al., 2022a). However, many recent techniques ameliorate this constraint, such as flavors of multi-query attention (Shazeer, 2019; Ainslie et al., 2023a), layer sparsity (de Jong et al., 2022a), speculative decoding (Leviathan et al., 2022; Chen et al., 2023), and others. Any model deployed in an environment where inference speed is important will likely employ one or more such techniques, such that FLOPs are a binding constraint. For the rest of this paper, we will measure computational cost in FLOPs; de Jong et al. (2023) contains analysis for how FLOPs and latency interact for LUMEN.

As we will show, GLIMMER represents a better quality-compute trade-off than LUMEN and FiD.

4 Experiments

4.1 Experimental setup

Model configuration GLIMMER is based on the T5.1.1 architecture (Raffel et al., 2020) like LUMEN, implemented in JAX (Heek et al., 2020), Flax (Heek et al., 2020) and Flaxformer. All models are initialized from public T5.1.1 checkpoints. FiD is fine-tuned according to the recipe from the original paper (Izacard and Grave, 2021). For LUMEN and GLIMMER, given proportion of live layers α, the memory encoder is initialized with the first 1 - α proportion of layers of the T5 encoder, and the live encoder is initialized with the last α proportion of layers of the T5 encoder. Main experiments use $\alpha = \frac{1}{3}$.

Fine-tuning For fine-tuning we use the Adafactor optimizer (Shazeer and Stern, 2018) with constant learning rate of 0.0001, batch size 128, and dropout rate 0.1 for all tasks. For multi-task training we sample uniformly from tasks. We allocate 48 tokens for the question and 304 tokens for each passage. In addition to the standard language modeling loss, reranking experiments use an auxiliary

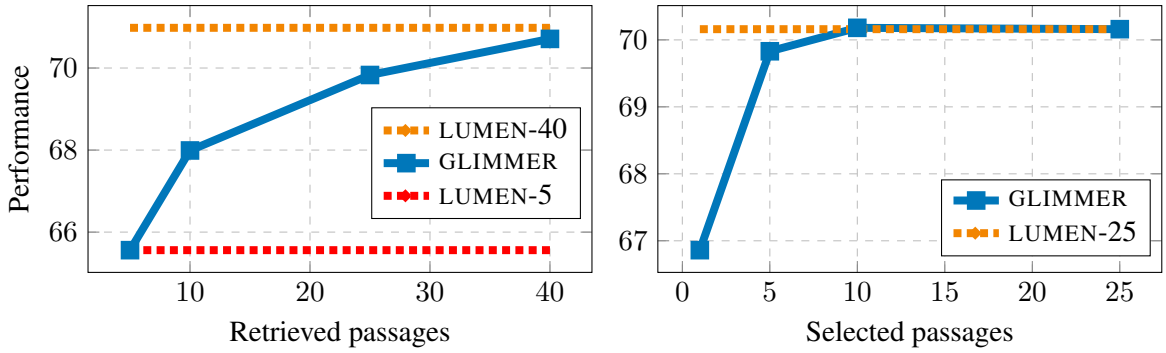

Figure 3: Average dev performance on KILT for GLIMMER-Large with live proportion $\frac{1}{3}$ and rerank proportion $\frac{1}{4}$ as a function of number of retrievals with 5 selected passages (left) and number of selected passages with 25 retrievals (right).

perplexity distillation loss with weight and temperature 1.0. We train until convergence and select the checkpoint with the highest performance on the dev set. We use greedy decoding for inference.

Data We train and evaluate on a subset of datasets from the KILT benchmark of knowledge-intensive tasks (Petroni et al., 2020). In particular, this includes question answering datasets Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), and HotPotQA (Yang et al., 2018), fact verification dataset FEVER (Thorne et al., 2018), and slot-filling datasets Zero Shot RE (Levy et al., 2017) and T-REx (ElSahar et al., 2018). We apply the relevance filtering procedure from Hofstätter et al. (2022) to ameliorate problems from imbalanced datasets.

Retrieval We employ the retrieval procedure from Hofstätter et al. (2022). Wikipedia is divided into chunks up to 200 words, and we retrieve the passages with the highest similarity score to the query, computed by a pre-trained GTR-Base model (Ni et al., 2021).

4.2 Main results

For our main results, we compare FiD, LUMEN (with updated architecture and multi-task training) and GLIMMER. Due to in-built reranking, GLIMMER processes passages more efficiently and can therefore retrieve more documents than LUMEN, which in turn can retrieve more documents than FiD. As Figure 2 shows, this efficiency translates into a higher quality and faster model, with GLIMMER outperforming LUMEN and FiD at faster speed.

4.3 Retrieval and reranking

The main results indicate that GLIMMER can achieve higher quality at lower cost than FiD and LUMEN by retrieving more passages initially and reranking to a much smaller number of passages. Here we investigate how different choices regarding retrieval and reranking affect the results.

Number of retrieved and selected passages Figure 3 shows how performance varies with the total number of retrieved passages and the number of selected passages after reranking. Performance strongly increases in the total number of retrieved passages, with sharply diminishing returns in the number of *selected* passages. These results indicate that the reranker effectively selects useful passages, such that the bottleneck is whether or not the relevant information is present in original retrieved passages.

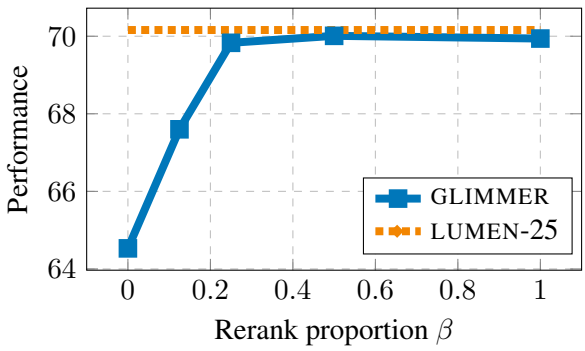

Figure 4: Average dev performance on KILT for GLIMMER-Large with live proportion $\frac{1}{3}$, 25 retrieved passages and 5 selected passages as a function of rerank proportion β. Baseline β is 0.25, equivalent to 2 reranking layers out of 8 total live layers.

The former intuition is further supported by Figure 4, as applying sufficient reranking layers almost

recovers the performance of using all 25 retrievals. On the other hand, some neural reranking with full interaction is clearly helpful, as using rerank proportion fewer than 0.25 (fewer than 2 reranking layers) strongly harms performance.

Interestingly, as shown in Figure 5, with a large number of retrievals, selection is sufficiently accurate that selecting more passages harms performance due to distraction from irrelevant context. The optimal number of selected passages is lower with more reranking layers, as the top ranked passages better capture all useful information.

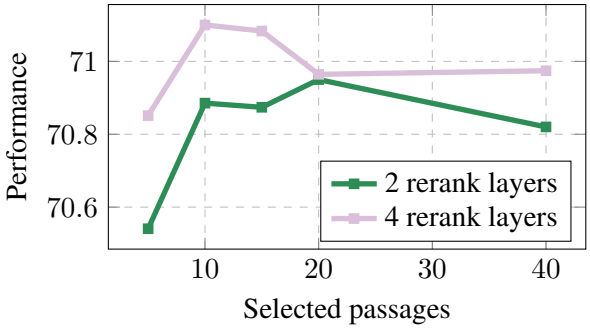

Figure 5: Average dev performance on KILT for GLIMMER-Large with live proportion $\frac{1}{3}$ with 40 retrievals as a function of number of selected passages.

Reranker	Performance
GLIMMER (shared)	69.8
Separate (from T5)	70.0
Separate (from scratch)	68.7

Table 1: Average performance on KILT dev sets for GLIMMER-Large with 25 retrieved and 5 selected passages for different configurations of the reranker: shared, separately initialized from T5, and separately initialized from scratch.

Separate reranker It is also informative to consider the effect of using the live encoder to perform the reranking, as opposed to a separate reranker. Table 1 compares performance of GLIMMER with using a separate reranker, initialized from T5 or trained from scratch. We note that using a separate reranker achieves comparable performance at the cost of a more complicated model, and additional memory and computation overhead. Initializing the reranker from pre-trained weights is important - attempting to learn reranking layers from scratch significantly lowers performance.

4.4 Multi-task training

The second major improvement in GLIMMER is sharing the memory and live encoder between tasks, and consequently training the memory encoder. We present experiments that attempt to disentangle the effects of these improvements.

Figure 6 demonstrates the effect of multi-task training by comparing performance on NQ between models trained only on NQ and models trained on KILT. To isolate the effect of multi-task training, we compare FiD and LUMEN, and train the memory for all models in this comparison. Multi-task training significantly benefits all models, but is disproportionately impactful for LUMEN, especially with lower live proportions. Figure 7 shows the difference between single and multi-task training as a function of live proportion, with multi-task performance leveling out earlier, further showing larger impact for smaller live proportion.

The late interaction that the live encoder is responsible for is rather different from its pre-training task, so it is intuitive that the live encoder would disproportionately benefit from increased size and diversity of data.

Multi-task training also enables learning a memory encoder. Table 2 shows that training the memory encoder is important for performance, which is expected as the pre-trained encoder is not designed to function as a memory encoder out of the box.

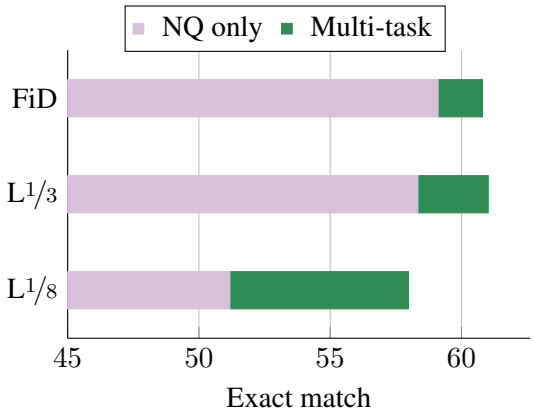

Figure 6: **Multi-task training disproportionately benefits LUMEN relative to FiD.** Exact match on Natural Questions dev set when trained only on Natural Questions vs on set of KILT tasks for FiD, GLIMMER-$\frac{1}{3}$ and GLIMMER-$\frac{1}{8}$ Large models.

4.5 Other ablations

There are a number of other interesting decisions in the GLIMMER architecture and training proce-

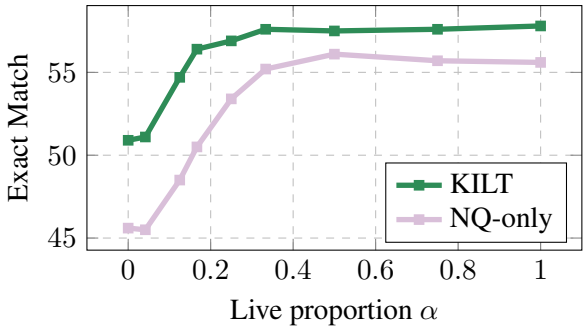

Figure 7: Performance on Natural Questions dev set for LUMEN-Large trained on KILT vs NQ-only as a function of live proportion.

Model	Performance
GLIMMER	69.8
Frozen memory	69.0

Table 2: **Training memory is a significant factor in strong GLIMMER performance.** Average performance on KILT dev sets for GLIMMER-Large with 25 retrieved and 5 selected passages, with and without training memory.

dure. Table 3 presents ablations of some of these decisions.

The original LUMEN implementation featured a separate question encoder, which was necessary because the memory encoder was not fine-tuned. Here, we update the memory encoder with multi-task training, so we opt to re-use the memory encoder for encoding the question, simplifying the architecture and reducing the number of parameters. We see that this simplification comes at a small cost in performance.

There are also a number of parameter choices regarding the reranking: the weight of the perplexity distillation loss, the temperature of the score and perplexity distributions, and the method for generating a reranking score. Over or under-weighting reranking loss leads to lower performance. However, using a lower temperature for the score and perplexity distributions does help - Izacard et al. (2022) argue that the effect of most individual passages on perplexity is small, and a lower temperature helps distinguish those differences. Finally, it appears that using the first token of each passage performs similarly to generating a score from mean-pooled representations.

Model	Performance
GLIMMER	69.8
Separate Qenc	70.0
PDist $\lambda = 0.1$	69.5
PDist $\lambda = 10$	69.5
PDist $\tau = 0.1$	70.1
PDist $\tau = 5$	69.4
Mean pool	69.8

Table 3: GLIMMER ablations: separate question encoder, different perplexity distillation loss weight, perplexity distillation temperature, and mean pool scoring method. Each model is Large size with 25 retrievals and 5 selected passages, evaluated on the KILT dev set.

5 Related Work

Retrieval augmentation (Izacard and Grave, 2021; Borgeaud et al., 2022; Lewis et al., 2020; Khandelwal et al., 2020; Guu et al., 2020) is a powerful technique to improve language model performance by augmenting the input with additional context. Our work is focused on improving the quality-compute trade-off for retrieval-augmented language models. It does so by unifying three lines of research: late-interaction memory, late-interaction reranking, and learning to retrieve. Our approach uses the architecture skeleton from Fusion-in-Decoder (Izacard and Grave, 2021), one of the most common retrieval augmented models. We employ multi-task training on KILT (Petroni et al., 2020) as in Hofstätter et al. (2022).

Memory Retrieval augmentation is expensive due to the additional context that needs to be processed by the language model. Memory models such as TOME (de Jong et al., 2022b), Memorizing Transformer (Wu et al., 2022a), and many others (Li et al., 2022; Zhong et al., 2022; Chen et al., 2022; Wu et al., 2022b; Yogatama et al., 2021; Bertsch et al., 2023) attempt to avoid this cost by pre-computing representations and storing them into a memory, such that representations can be retrieved directly rather than processed on the fly. However, such approaches sacrifice quality as memory representations are not conditioned on each individual input (Li et al., 2022; de Jong et al., 2023). *Late-interaction memory* (de Jong et al., 2023; Milbauer et al., 2023) improves the quality of memory approaches by only partially pre-computing retrieval representations, and performing some interaction between memory and input

on the fly. In particular, our work is very closely based on LUMEN (de Jong et al., 2023).

Reranking Like the language model itself, retrieval procedures face a trade-off between expensive online ranking with full interaction (Chen et al., 2020) and the more common dual encoder approaches such as DPR (Karpukhin et al., 2020) and GTR (Ni et al., 2021) that scores based on inner product similarity with a corpus of pre-computed passage representations.

Often different models for retrieval are applied in a pipeline approach, with an initial cheap scoring model followed by a more powerful and expensive reranker (Mao et al., 2021; Wang et al., 2018; Yu et al., 2022). Many rerankers also make use of late interaction to obtain a good trade-off between ranking quality and speed, such as COLBERT (Khattab and Zaharia, 2020; Santhanam et al., 2022), PreTTR (MacAvaney et al., 2020), SDR (Cohen et al., 2022), and Poly-encoders (Humeau et al., 2020). GLIMMER combines late-interaction memory and reranking into a single model, sharing the pre-computed representations for both use cases.

Learning to retrieve Retrieval models are often trained with supervised data (Karpukhin et al., 2020; Ni et al., 2021), using gold retrievals from datasets such as MS-MARCO (Nguyen et al., 2016) or TREC CAR (Dietz et al., 2018). When selecting passage to use for retrieval-augmented generation, we have an additional signal, namely which passages are most helpful for the reader model. A number of existing works use this signal to improve retrieval (Guu et al., 2020; Sachan et al., 2021; Jiang et al., 2022; Sachan et al., 2021; Izacard et al., 2022). We follow ATLAS (Izacard et al., 2022) and employ perplexity distillation to train our reranker to select passages that help lower reader model perplexity.

6 Conclusion

Retrieval-augmented language models are powerful but slow in inference, while pre-computed memory-augmented models are fast at the cost of quality. Hybrid late-interaction models such as LUMEN present a good quality-compute trade-off. We introduce GLIMMER, an improved late-interaction model that also incorporates learned end-to-end reranking and multi-task training to achieve an even better trade-off. GLIMMER achieves strong gains in quality at faster speeds compared to LUMEN and FiD on the KILT benchmark of knowledge-intensive tasks.

Acknowledgements

We thank Luke Vilnis, Tania Bedrax-Weiss and others at Google Research for insightful comments and discussion.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023a. GQA: training generalized multi-query transformer models from multi-head checkpoints. *CoRR*, abs/2305.13245.

Joshua Ainslie, Tao Lei, Michiel de Jong, Santiago Ontañón, Siddhartha Brahma, Yury Zemlyanskiy, David C. Uthus, Mandy Guo, James Lee-Thorp, Yi Tay, Yun-Hsuan Sung, and Sumit Sanghai. 2023b. Colt5: Faster long-range transformers with conditional computation. *CoRR*, abs/2303.09752.

Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. 2023. Unlimiformer: Long-range transformers with unlimited length input. *CoRR*, abs/2305.01625.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. 2022. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *CoRR*, abs/2302.01318.

Dongmei Chen, Sheng Zhang, Xin Zhang, and Kaijing Yang. 2020. Cross-lingual passage re-ranking with alignment augmented multilingual BERT. *IEEE Access*, 8:213232–213243.

Wenhu Chen, Pat Verga, Michiel de Jong, John Wieting, and William W. Cohen. 2022. Augmenting pre-trained language models with qa-memory for open-domain question answering. *CoRR*, abs/2204.04581.

Nachshon Cohen, Amit Portnoy, Besnik Fetahu, and Amir Ingber. 2022. SDR: efficient neural re-ranking

using succinct document representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pages 6624–6637. Association for Computational Linguistics.

Michiel de Jong, Yury Zemlyanskiy, Joshua Ainslie, Nicholas FitzGerald, Sumit Sanghai, Fei Sha, and William Cohen. 2022a. FiDO: Fusion-in-decoder optimized for stronger performance and faster inference. *arXiv preprint arXiv:2212.08153*.

Michiel de Jong, Yury Zemlyanskiy, Nicholas FitzGerald, Joshua Ainslie, Sumit Sanghai, Fei Sha, and William W. Cohen. 2023. Pre-computed memory or on-the-fly encoding? A hybrid approach to retrieval augmentation makes the most of your compute. *CoRR*, abs/2301.10448.

Michiel de Jong, Yury Zemlyanskiy, Nicholas FitzGerald, Fei Sha, and William W. Cohen. 2022b. Mention memory: incorporating textual knowledge into transformers through entity mention attention. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

Laura Dietz, Ben Gamari, Jeff Dalton, and Nick Craswell. 2018. TREC complex answer retrieval overview. In *Proceedings of the Twenty-Seventh Text REtrieval Conference, TREC 2018, Gaithersburg, Maryland, USA, November 14-16, 2018*, volume 500-331 of *NIST Special Publication*. National Institute of Standards and Technology (NIST).

Hady ElSahar, Pavlos Vougiouklis, Arslen Remaci, Christophe Gravier, Jonathon S. Hare, Frédérique Laforest, and Elena Simperl. 2018. T-rex: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation, LREC 2018, Miyazaki, Japan, May 7-12, 2018*. European Language Resources Association (ELRA).

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: retrieval-augmented language model pre-training. *CoRR*, abs/2002.08909.

Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. 2020. Flax: A neural network library and ecosystem for JAX.

Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2022. Multi-task retrieval-augmented text generation with relevance sampling. *CoRR*, abs/2207.03030.

Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. 2020. Poly-encoders: Architectures and pre-training strategies for fast and accurate multi-sentence scoring. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL 2021, Online, April 19 - 23, 2021*, pages 874–880. Association for Computational Linguistics.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. Few-shot learning with retrieval augmented language models. *CoRR*, abs/2208.03299.

Zhengbao Jiang, Luyu Gao, Zhiruo Wang, Jun Araki, Haibo Ding, Jamie Callan, and Graham Neubig. 2022. Retrieval as attention: End-to-end learning of retrieval and reading within a single transformer. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 2336–2349. Association for Computational Linguistics.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 1601–1611. Association for Computational Linguistics.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick S. H. Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 6769–6781. Association for Computational Linguistics.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Generalization through memorization: Nearest neighbor language models. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 39–48. ACM.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: a benchmark for question answering research. *Trans. Assoc. Comput. Linguistics*, 7:452–466.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2022. Fast inference from transformers via speculative decoding. *CoRR*, abs/2211.17192.

Omer Levy, Minjoon Seo, Eunsol Choi, and Luke Zettlemoyer. 2017. Zero-shot relation extraction via reading comprehension. In *Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017), Vancouver, Canada, August 3-4, 2017*, pages 333–342. Association for Computational Linguistics.

Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Zonglin Li, Ruiqi Guo, and Sanjiv Kumar. 2022. Decoupled context processing for context augmented language modeling. *CoRR*, abs/2210.05758.

Sean MacAvaney, Franco Maria Nardini, Raffaele Perego, Nicola Tonellotto, Nazli Goharian, and Ophir Frieder. 2020. Efficient document re-ranking for transformers by precomputing term representations. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 49–58. ACM.

Yuning Mao, Pengcheng He, Xiaodong Liu, Yelong Shen, Jianfeng Gao, Jiawei Han, and Weizhu Chen. 2021. Reader-guided passage reranking for open-domain question answering. In *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, volume ACL/IJCNLP 2021 of *Findings of ACL*, pages 344–350. Association for Computational Linguistics.

Jeremiah Lev Milbauer, Annie Louis, Javad Hosseini, Alex Fabrikant, Don Metzler, and Tal Schuster. 2023. Lait: Efficient multi-segment encoding in transformers with layer-adjustable interaction. In *Proceedings of the Association for Computational Linguistics: ACL 2023*.

Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A human generated machine reading comprehension dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016*, volume 1773 of *CEUR Workshop Proceedings*. CEUR-WS.org.

Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y. Zhao, Yi Luan, Keith B. Hall, Ming-Wei Chang, and Yinfei Yang. 2021. Large dual encoders are generalizable retrievers. *CoRR*, abs/2112.07899.

Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick S. H. Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2020. KILT: a benchmark for knowledge intensive language tasks. *CoRR*, abs/2009.02252.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67.

Devendra Singh Sachan, Siva Reddy, William L. Hamilton, Chris Dyer, and Dani Yogatama. 2021. End-to-end training of multi-document reader and retriever for open-domain question answering. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 25968–25981.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, pages 3715–3734. Association for Computational Linguistics.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. *arXiv preprint arXiv:2207.07061*.

Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*.

Noam Shazeer and Mitchell Stern. 2018. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 4603–4611. PMLR.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, pages 809–819. Association for Computational Linguistics.

Neeraj Varshney, Man Luo, and Chitta Baral. 2022. Can open-domain QA reader utilize external knowledge efficiently like humans? *CoRR*, abs/2211.12707.

Shuohang Wang, Mo Yu, Xiaoxiao Guo, Zhiguo Wang, Tim Klinger, Wei Zhang, Shiyu Chang, Gerry Tesauro, Bowen Zhou, and Jing Jiang. 2018. R^3: Reinforced ranker-reader for open-domain question answering. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, pages 5981–5988. AAAI Press.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. 2022a. Memorizing transformers. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

Yuxiang Wu, Yu Zhao, Baotian Hu, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2022b. An efficient memory-augmented transformer for knowledge-intensive NLP tasks. *CoRR*, abs/2210.16773.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 2369–2380. Association for Computational Linguistics.

Dani Yogatama, Cyprien de Masson d'Autume, and Lingpeng Kong. 2021. Adaptive semiparametric language models. *Trans. Assoc. Comput. Linguistics*, 9:362–373.

Donghan Yu, Chenguang Zhu, Yuwei Fang, Wenhao Yu, Shuohang Wang, Yichong Xu, Xiang Ren, Yiming Yang, and Michael Zeng. 2022. Kg-fid: Infusing knowledge graph in fusion-in-decoder for open-domain question answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pages 4961–4974. Association for Computational Linguistics.

Zexuan Zhong, Tao Lei, and Danqi Chen. 2022. Training language models with memory augmentation. *CoRR*, abs/2205.12674.